

Geometric Background

Fundamentals of Robotics
University of Trento

TEACHER: Luigi Palopoli



Matlab Implementation of rotations

Still on Rotation Matrices

- Composition

- Parametrisation

- Quaternions

Homogeneous Transformations

- Matlab Examples for Homogeneous Transformations

Some questions for you

- Fundamental Rotation Matrices

- Composition of Rotations

- Parametrisation of Rotations

- Axis–Angle Representation

- Quaternions

- Homogeneous Transformations

Fundamental Rotation Matrices

We've seen how to define a rotation matrix about the z-axis.

$$R_z(\alpha) = \begin{bmatrix} \cos(\alpha) & -\sin(\alpha) & 0 \\ \sin(\alpha) & \cos(\alpha) & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Matlab translation: (????)

Fundamental Rotation Matrices

We've seen how to define a rotation matrix about the z-axis.

$$R_z(\alpha) = \begin{bmatrix} \cos(\alpha) & -\sin(\alpha) & 0 \\ \sin(\alpha) & \cos(\alpha) & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Matlab translation: (????)

zRot

```
1 % Function that returns the Rotation about z axis
2 function [Rz] = zRot(alpha)
3     Rz = [
4         cos(alpha), -sin(alpha), 0;
5         sin(alpha), cos(alpha), 0;
6         0, 0, 1
7     ];
8 end
```

We can do the same for the other fundamental rotation matrices.



Fundamental Rotation Matrices

Rotation about the y-axis

Fundamental Rotation Matrices

Rotation about the y-axis

yRot

```
1 % Function that returns the Rotation about z axis
2 function [Ry] = yRot(beta)
3     Ry = [
4         cos(beta), 0, sin(beta);
5         0, 1, 0;
6         -sin(beta), 0, cos(beta)
7     ];
8 end
```

Rotation about the x-axis

Fundamental Rotation Matrices

Rotation about the y-axis

yRot

```
1 % Function that returns the Rotation about z axis
2 function [Ry] = yRot(beta)
3     Ry = [
4         cos(beta), 0, sin(beta);
5         0, 1, 0;
6         -sin(beta), 0, cos(beta)
7     ];
8 end
```

Rotation about the x-axis

xRot

```
1 % Function that returns the Rotation about z axis
2 function [Rx] = xRot(gamma)
3     Rx = [
4         1, 0, 0;
5         0, cos(gamma), -sin(gamma);
6         0, sin(gamma), cos(gamma)
7     ];
8 end
```

Draw a Reference Frame

Let's now draw and rotate a frame.

The drawing can be created using the following script:

scriptDraw

```

1  close all;                                %% CLOSING ALL WINDOWS
2  view(3);                                  %% Uses the default line of sight
    for 2D or 3D plots
3  daspect([1 1 1]);                        %% Sets the three scales of the axes
4  plot3([0,1],[0,0],[0,0], 'Color', [1,0,0]); %% Plots the first axis (color red)
5  hold on                                   %% Holds the image in the plot
6  plot3([0,0],[0,1],[0,0], 'Color', [0,1,0]); %% Plots the y axis
7  plot3([0,0],[0,0],[0,1], 'Color', [0,0,1]); %% Plots the z axis
8
9  v1 = zRot(pi/4)*[1;0;0]                  %% First vector oriented along the x
    axis rotated by 45 deg around z
10 v2 = zRot(pi/4)*[0;1;0]                  %% Second vector oriented along the
    y axis rotated by 45deg around z
11 v3 = zRot(pi/4)*[0;0;1]                  %% Third vector in oriented along
    the z axis rotated by 45deg around z
12
13 %% Plot the three rotated vectors
14 plot3([0,v1(1)],[0,v1(2)],[0,v1(3)], 'Color', [1,0,0], 'LineStyle', '—');
15 plot3([0,v2(1)],[0,v2(2)],[0,v2(3)], 'Color', [0,1,0], 'LineStyle', '—');
16 plot3([0,v3(1)],[0,v3(2)],[0,v3(3)], 'Color', [0,0,1], 'LineStyle', '—');

```

Matlab Implementation of rotations

Still on Rotation Matrices

- Composition

- Parametrisation

- Quaternions

Homogeneous Transformations

- Matlab Examples for Homogeneous Transformations

Some questions for you

- Fundamental Rotation Matrices

- Composition of Rotations

- Parametrisation of Rotations

- Axis–Angle Representation

- Quaternions

- Homogeneous Transformations



Outline

Matlab Implementation of rotations

Still on Rotation Matrices

Composition

Parametrisation

Quaternions

Homogeneous Transformations

Matlab Examples for Homogeneous Transformations

Some questions for you

Fundamental Rotation Matrices

Composition of Rotations

Parametrisation of Rotations

Axis–Angle Representation

Quaternions

Homogeneous Transformations

Composition of Rotation Matrices

Let us start with an example.

- ▶ Consider three different frames sharing the same origin: $O - x_0y_0z_0$, $O - x_1y_1z_1$, $O - x_2y_2z_2$
- ▶ Consider a point P with coordinates $\mathbf{p}^0$, $\mathbf{p}^1$, and $\mathbf{p}^2$ in the three frames.
- ▶ Let R_i^j denote the rotation matrix of Frame i with respect to Frame j .
- ▶ We have:

$$\mathbf{p}^1 = R_2^1 \mathbf{p}^2$$

$$\mathbf{p}^0 = R_1^0 \mathbf{p}^1$$

$$\mathbf{p}^0 = R_2^0 \mathbf{p}^2$$

- ▶ By simple substitution and comparison,

$$\mathbf{p}^0 = R_1^0 R_2^1 \mathbf{p}^2$$

$$\mathbf{p}^1 = R_2^1 \mathbf{p}^2 \rightarrow$$

$$R_2^0 = R_1^0 R_2^1$$

Composition of Rotation Matrices

We can interpret the relation $R_2^0 = R_1^0 R_2^1$ as being obtained in the two following steps:

1. First, rotate $O - x_0 y_0 z_0$ so as to align it with frame $O - x_1 y_1 z_1$ (matrix R_1^0).
2. Then, rotate again so as to align frame $O - x_1 y_1 z_1$ with $O - x_2 y_2 z_2$ (matrix R_2^1).

Generally speaking:

Composition of Rotations

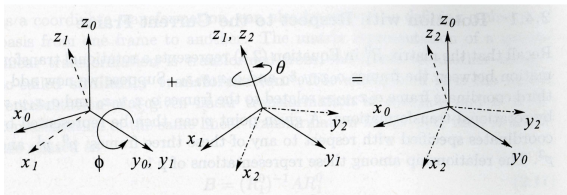
A rotation can be expressed as a sequence of partial rotations, each one defined with respect to the previous one. The frame from which the rotation occurs is termed the *current frame*. The sequence is obtained by post-multiplying the matrices associated with each rotation.

In view of the properties of rotation matrices, we have:

$$R_i^j = (R_j^i)^{-1} = (R_j^i)^T$$

Composition of Rotation Matrices - Example

An example of a sequence of rotations about axes in *the current frame* is below.



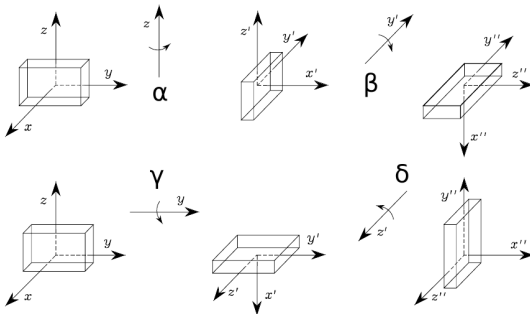
- ▶ We first rotate by ϕ about the current y-axis.
- ▶ Then we rotate by θ about the current z-axis.

Result:

$$R_2^0 = R_y(\phi)R_z(\theta).$$

Composition of Rotation Matrices - Another example

An example of a sequence of rotations about axes in *the current frame* is below.



The resulting rotation matrix is:

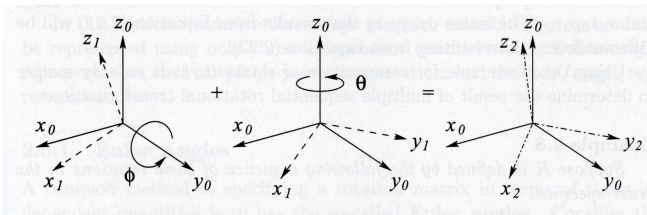
$$R_4^0 = R_z(\alpha)R_y(\beta)R_y(\gamma)R_z(\delta).$$

The order is absolutely important!

$$R_z(\alpha)R_y(\beta)R_y(\gamma)R_z(\delta) \neq R_y(\gamma)R_z(\delta)R_z(\alpha)R_y(\beta).$$

Rotation about fixed axes

- What if we want to specify the sequence of rotations about fixed axes
- An example of a sequence of rotations *in a fixed frame* is below



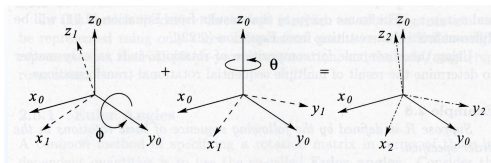
Rotation about fixed axes

- ▶ To see how rotations can be composed about a fixed axis, let's use a simple fact:
- ▶ Consider two rotated frames $O - x_0y_0z_0$ and $O - x_1y_1z_1$.
- ▶ For a generic point P we have: $p^0 = R_1^0 p^1$
- ▶ Consider a generic linear application A defined in the $O - x_0y_0z_0$ frame.
- ▶ For a generic point P , its transformed point we have:

$$q^0 = Ap^0$$

$$q^1 = R_0^1 Ap^0 = R_0^1 AR_1^0 p^1 = (R_1^0)^{-1} AR_1^0 p^1$$

Rotation about fixed axes - back to the example



We have two rotations:

- ▶ The first rotation is $R_1^0 = R_y(\phi)$ around the y_0 -axis.
- ▶ The rotation $\bar{R}_2^1$, which happens around the z_0 -axis (not the z_1 -axis as in the case of the current frame).

$$R_2^0 = R_1^0 \bar{R}_2^1.$$

We know how the latter is done in the current axes, so we can use the result above:

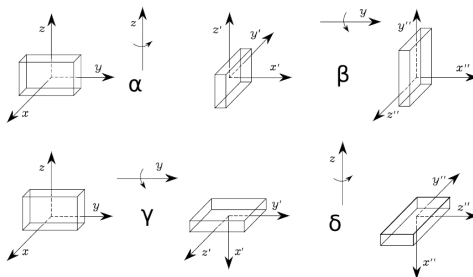
$$\bar{R}_2^1 = (R_1^0)^{-1} R_z(\theta) R_1^0,$$

which leads to

$$R_2^0 = R_1^0 \bar{R}_2^1 = R_1^0 (R_1^0)^{-1} R_z(\theta) R_1^0 = R_z(\theta) R_y(\phi)$$

Rotation about fixed axes - back to the example

We can conclude that when the composition of rotations happens with fixed axes, we have to pre-multiply instead of post-multiplying (as is the case for rotations in the current frame). Consider the example:



We have:

$$R_4^0 = R_z(\delta)R_y(\gamma)R_y(\beta)R_z(\alpha)$$

Rotation about current axes - Example

Consider the rotation composed as follows:

1. A rotation of θ about the current x-axis.
2. A rotation of ϕ around the current z-axis.

The result is:

$$R_2^0 = R_x(\theta)R_z(\phi).$$

A point expressed as p^2 in the frame $O - x_2y_2z_2$ corresponds to $p^0 = R_2^0 p^2$ in the initial frame.

A More Complex Example

Consider the rotation composed as follows:

1. A rotation of θ about the current x-axis.
2. A rotation of ϕ around the current z-axis.
3. A rotation of α about the *fixed* z_0 -axis.

Following the same procedure as above, we have:

The result is:

$$R_3^0 = R_1^0 R_2^1 \bar{R}_3^2 = R_x(\theta) R_z(\phi) \bar{R}_3^2.$$

where $\bar{R}_3^2$ denotes the notation with respect to the initial fixed axes. The effect of the rotation about the fixed frame in the current frame

$O - x_2 y_2 z_2$ is given by:

$$\bar{R}_3^2 = (R_1^0 R_2^1)^{-1} R_z(\alpha) R_1^0 R_2^1 = (R_x(\theta) R_z(\phi))^{-1} R_z(\alpha) R_x(\theta) R_z(\phi)$$

which leads us to

$$R_3^0 = R_z(\alpha) R_x(\theta) R_z(\phi).$$

The Final Example

Consider the rotation composed as follows:

1. A rotation of θ about the current x-axis.
2. A rotation of ϕ around the current z-axis.
3. A rotation of α about the *fixed* z_0 -axis.
4. A rotation of β around the current y-axis.
5. A rotation of δ about the fixed x_0 -axis.

Following the same procedure that we have seen before, we get:

$$R_r^0 = R_x(\delta)R_z(\alpha)R_x(\theta)R_z(\phi)R_y(\beta).$$

Please verify this yourself with paper and a pencil.

Matlab Implementation of rotations

Still on Rotation Matrices

Composition

Parametrisation

Quaternions

Homogeneous Transformations

Matlab Examples for Homogeneous Transformations

Some questions for you

Fundamental Rotation Matrices

Composition of Rotations

Parametrisation of Rotations

Axis–Angle Representation

Quaternions

Homogeneous Transformations

Parametrisation of Rotation Matrices

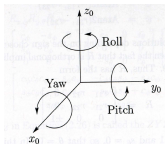
- ▶ A 3D rotation matrix has 9 elements, but in fact a rotation is completely specified by three parameters.
- ▶ Generally speaking, a rotation is an element of $SO(m)$ that is completely specified by $m(m-1)/2$ parameters.
 - ▶ 1 parameter for $SO(2)$
 - ▶ 3 parameters for $SO(3)$
- ▶ A 3D rotation is completely specified by three rotations about different axes, such that no two adjacent rotations are about the same axis.
- ▶ As a result, there are 12 parametrisations with three rotations:

3 ×	Any of the three axes is acceptable for the first rotation
2 ×	Rule out the axis used for the first rotation
2 =	Rule out the axis used for the second rotation
	12

- ▶ We will mention the most common ones.

Euler Angles

- ▶ Euler angles are the angles related to three rotations about the current axis.
- ▶ A very popular convention is the use of XYZ rotations about fixed axes, which is known in the aeronautical world as roll-pitch-yaw because these rotations denote the change in attitude of an aircraft with respect to three fixed axes.



- ▶ Since the rotation is about fixed axes, we have to pre-multiply:

$$R = R_z(\phi)R_y(\theta)R_x(\psi)$$

- ▶ ψ is the roll angle, θ is the pitch angle, ϕ is the yaw angle.

Euler - Matlab

We can create any rotation matrix by specifying three angles. One way is the so-called triple of Euler angles (three rotations about the current axes: ZYZ).

$$R(\phi, \theta, \psi) = R_z(\phi)R_{y'}(\theta)R_{z''}(\psi)$$

The corresponding Matlab listing is:

eulerRot

```
1 function [R] = eulerRot(phi, theta, psi)
2
3 R = zRot(phi)*yRot(theta)*zRot(psi);
4
5 end
```

The function can be tested through the following script:

scriptDrawEuler

```
1 close all;
2
3 view(3);
4 daspect([1 1 1]);
5 plot3([0,1],[0,0],[0,0], 'Color',[1,0,0]);
6 hold on
7 plot3([0,0],[0,1],[0,0], 'Color',[0,1,0]);
8 plot3([0,0],[0,0],[0,1], 'Color',[0,0,1]);
9
10 R = eulerRot(pi/6, pi/6, 0);
11
12 v1 = R*[1;0;0]
13 v2 = R*[0;1;0]
14 v3 = R*[0;0;1]
15
16 plot3([0,v1(1)],[0,v1(2)],[0,v1(3)], 'Color',[1,0,0], 'LineStyle','—');
17 plot3([0,v2(1)],[0,v2(2)],[0,v2(3)], 'Color',[0,1,0], 'LineStyle','—');
18 plot3([0,v3(1)],[0,v3(2)],[0,v3(3)], 'Color',[0,0,1], 'LineStyle','—');
```

RPY Angles

- ▶ Introduce the notation $c_\alpha = \cos(\alpha)$, $s_\alpha = \sin(\alpha)$.
- ▶ It is easy to see that:

$$R = \begin{bmatrix} c_\phi c_\theta & c_\phi s_\theta s_\psi - s_\phi c_\psi & c_\phi s_\theta c_\psi + s_\phi s_\psi \\ s_\phi c_\theta & s_\phi s_\theta s_\psi + c_\phi c_\psi & s_\phi s_\theta c_\psi - c_\phi s_\psi \\ -s_\theta & c_\theta s_\psi & c_\theta c_\psi \end{bmatrix}$$

- ▶ Given a Rotation matrix $R = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix}$, we can solve the inverse problem (try this as an exercise) and find two solutions:

$$\begin{aligned} \theta \in (-\pi/2, \pi/2) &\rightarrow \begin{cases} \phi = \text{atan2}(r_{21}, r_{11}) \\ \theta = \text{atan2}\left(-r_{31}, \sqrt{r_{32}^2 + r_{33}^2}\right) \\ \psi = \text{atan2}(r_{32}, r_{33}) \end{cases} \\ \theta \in (\pi/2, 3\pi/2) &\rightarrow \begin{cases} \phi = \text{atan2}(-r_{21}, -r_{11}) \\ \theta = \text{atan2}\left(-r_{31}, -\sqrt{r_{32}^2 + r_{33}^2}\right) \\ \psi = \text{atan2}(-r_{32}, -r_{33}) \end{cases} \end{aligned}$$



RPY Angles - Gimbal Lock

- ▶ An important feature of RPY, and generally of Euler-based representations, is that the three angles act as three independent degrees of freedom.

RPY Angles - Gimbal Lock

- ▶ An important feature of RPY, and generally of Euler-based representations, is that the three angles act as three independent degrees of freedom.
- ▶ At a pitch angle of $\pm \frac{\pi}{2}$, the roll and yaw rotation axes become aligned. This results in the loss of a degree of freedom.

RPY Angles - Gimbal Lock

- ▶ An important feature of RPY, and generally of Euler-based representations, is that the three angles act as three independent degrees of freedom.
- ▶ At a pitch angle of $\pm \frac{\pi}{2}$, the roll and yaw rotation axes become aligned. This results in the loss of a degree of freedom.
- ▶ This means a rotation around one axis can be replicated by a rotation around the other. Indeed, when $\theta = \frac{\pi}{2}$:

$$\begin{aligned}
 R &= \begin{bmatrix} c_\phi c_\theta & c_\phi s_\theta s_\psi - s_\phi c_\psi & c_\phi s_\theta c_\psi + s_\phi s_\psi \\ s_\phi c_\theta & s_\phi s_\theta s_\psi + c_\phi c_\psi & s_\phi s_\theta c_\psi - c_\phi s_\psi \\ -s_\theta & c_\theta s_\psi & c_\theta c_\psi \end{bmatrix} \\
 &= \begin{bmatrix} 0 & c_\phi s_\psi - s_\phi c_\psi & c_\phi c_\psi + s_\phi s_\psi \\ 0 & s_\phi s_\psi + c_\phi c_\psi & s_\phi c_\psi - c_\phi s_\psi \\ -1 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & s(\phi+\psi) & c(\phi+\psi) \\ 0 & c(\phi+\psi) & -s(\phi+\psi) \\ -1 & 0 & 0 \end{bmatrix}
 \end{aligned}$$

RPY Angles - The Gimbal Lock (1)

- From $R = \begin{bmatrix} 0 & s(\phi+\psi) & c(\phi+\psi) \\ 0 & c(\phi+\psi) & -s(\phi+\psi) \\ -1 & 0 & 0 \end{bmatrix}$, we can see that ϕ and ψ do not act independently. All rotations that can be achieved with a combination of ϕ and ψ can be accomplished by a single rotation of angle $(\phi + \psi)$.

RPY Angles - The Gimbal Lock (1)

- ▶ From $R = \begin{bmatrix} 0 & s(\phi+\psi) & c(\phi+\psi) \\ 0 & c(\phi+\psi) & -s(\phi+\psi) \\ -1 & 0 & 0 \end{bmatrix}$, we can see that ϕ and ψ do not act independently. All rotations that can be achieved with a combination of ϕ and ψ can be accomplished by a single rotation of angle $(\phi + \psi)$.
- ▶ The inverse mapping from the rotation matrix to the RPY angles is singular at this point, meaning there are infinitely many combinations of ϕ and ψ that produce the same rotation matrix.

RPY Angles - The Gimbal Lock (1)

- ▶ From $R = \begin{bmatrix} 0 & s(\phi+\psi) & c(\phi+\psi) \\ 0 & c(\phi+\psi) & -s(\phi+\psi) \\ -1 & 0 & 0 \end{bmatrix}$, we can see that ϕ and ψ do not act independently. All rotations that can be achieved with a combination of ϕ and ψ can be accomplished by a single rotation of angle $(\phi + \psi)$.
- ▶ The inverse mapping from the rotation matrix to the RPY angles is singular at this point, meaning there are infinitely many combinations of ϕ and ψ that produce the same rotation matrix.
- ▶ This problem has profound reasons. It's like creating a 2D map from a sphere. When you flatten it, the poles become lines; a single point on the sphere is mapped to an infinite number of points.

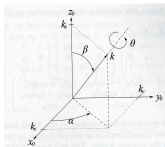


RPY Angles - The Gimbal Lock (1)

- From $R = \begin{bmatrix} 0 & s(\phi+\psi) & c(\phi+\psi) \\ 0 & c(\phi+\psi) & -s(\phi+\psi) \\ -1 & 0 & 0 \end{bmatrix}$, we can see that ϕ and ψ do not act independently. All rotations that can be achieved with a combination of ϕ and ψ can be accomplished by a single rotation of angle $(\phi + \psi)$.
- The inverse mapping from the rotation matrix to the RPY angles is singular at this point, meaning there are infinitely many combinations of ϕ and ψ that produce the same rotation matrix.
- This problem has profound reasons. It's like creating a 2D map from a sphere. When you flatten it, the poles become lines; a single point on the sphere is mapped to an infinite number of points.
- This lack of invertibility can cause a significant problem: in the singular configuration, small changes in the physical orientation can produce discontinuous jumps in the calculated RPY angles. This is a major issue for physical systems like robots, which require smooth and continuous control.

Axis and rotation

- ▶ We can adopt a redundant parametrisation using an axis ($\mathbf{k} = [k_x, k_y, k_z]$ with $|\mathbf{k}| = 1$) and an angle (θ), resulting in four parameters overall.



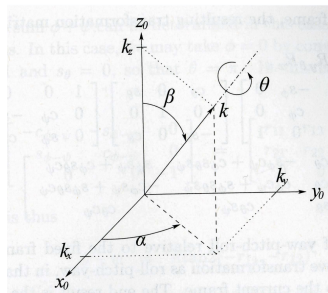
- ▶ The rotation $R_{\mathbf{k}}(\theta)$ can be found by transforming the coordinates.
- ▶ We can find a new frame $O - x_1y_1z_1$, with z_1 coincident with $\mathbf{k}$. We rotate $O - x_0y_0z_0$ by α about the z -axis and by β around the y -axis.
- ▶ Clearly $R_1^0 = R_z(\alpha)R_y(\beta)$. If a transformation A (e.g., a rotation) is expressed in the frame $O - x_1y_1z_1$, as in $q^1 = Ap^1$, we know that:

$$q^0 = (R_0^1)^{-1}AR_0^1p^0 = R_z(\alpha)R_y(\beta)A(R_z(\alpha)R_y(\beta))^{-1}$$

which leads to:

$$R_{\mathbf{k}}(\theta) = R_z(\alpha)R_y(\beta)R_z(\theta)R_y(-\beta)R_z(-\alpha)$$

Axis and rotation



Using some trigonometry, we can find the α and β angles.

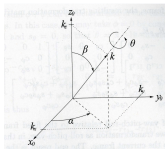
$$\sin(\alpha) = \frac{k_y}{\sqrt{k_x^2 + k_y^2}} \quad \cos(\alpha) = \frac{k_x}{\sqrt{k_x^2 + k_y^2}}$$

$$\sin(\beta) = \frac{\sqrt{k_x^2 + k_y^2}}{\sqrt{k_x^2 + k_y^2 + k_z^2}} \quad \cos(\beta) = \frac{k_z}{\sqrt{k_x^2 + k_y^2 + k_z^2}}$$

and

$$R_{\mathbf{k}}(\theta) = \begin{bmatrix} k_x^2(1 - c_\theta) + c_\theta & k_x k_y(1 - c_\theta) - k_z s_\theta & k_x k_z(1 - c_\theta) + k_y s_\theta \\ k_x k_y(1 - c_\theta) + k_z s_\theta & k_y^2(1 - c_\theta) + c_\theta & k_y k_z(1 - c_\theta) - k_x s_\theta \\ k_x k_z(1 - c_\theta) - k_y s_\theta & k_y k_z(1 - c_\theta) + k_x s_\theta & k_z^2(1 - c_\theta) + c_\theta \end{bmatrix}$$

Axis and Rotation

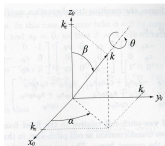


- We've specified a rotation matrix using four parameters ($\mathbf{k}$ and θ), but there's also a constraint ($k_x^2 + k_y^2 + k_z^2 = 1$), so the system actually has three degrees of freedom.
- From the Rodrigues' formula:

$$R_{\mathbf{k}}(\theta) = \begin{bmatrix} k_x^2(1 - c_\theta) + c_\theta & k_x k_y(1 - c_\theta) - k_z s_\theta & k_x k_z(1 - c_\theta) + k_y s_\theta \\ k_x k_y(1 - c_\theta) + k_z s_\theta & k_y^2(1 - c_\theta) + c_\theta & k_y k_z(1 - c_\theta) - k_x s_\theta \\ k_x k_z(1 - c_\theta) - k_y s_\theta & k_y k_z(1 - c_\theta) + k_x s_\theta & k_z^2(1 - c_\theta) + c_\theta \end{bmatrix}$$

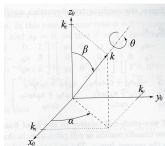
- We can see that $R_{-\mathbf{k}}(-\theta) = R_{\mathbf{k}}(\theta)$.
- If we invert both the axis and the angle, we get the same rotation.

Axis and Rotation



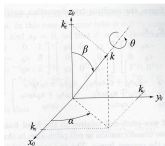
- We've seen that the axis-angle parameterisation isn't *globally invertible*.

Axis and Rotation



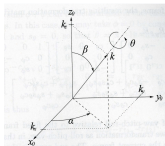
- We've seen that the axis-angle parameterisation isn't *globally invertible*.
- However, it is *locally invertible* (to be precise, it's a local homeomorphism). This means, unlike with RPY angles, there are no singularities such as Gimbal Lock.

Axis and Rotation



- ▶ We've seen that the axis-angle parameterisation isn't *globally invertible*.
- ▶ However, it is *locally invertible* (to be precise, it's a local homeomorphism). This means, unlike with RPY angles, there are no singularities such as Gimbal Lock.
- ▶ The problem, compared to RPY angles, is that this parameterisation isn't very intuitive.

Axis and Rotation



- ▶ We've seen that the axis-angle parameterisation isn't *globally invertible*.
- ▶ However, it is *locally invertible* (to be precise, it's a local homeomorphism). This means, unlike with RPY angles, there are no singularities such as Gimbal Lock.
- ▶ The problem, compared to RPY angles, is that this parameterisation isn't very intuitive.
- ▶ Unlike using rotation matrices directly, it's not straightforward to understand how to analytically compose or invert subsequent rotations using the axis-angle representation.



Outline

Matlab Implementation of rotations

Still on Rotation Matrices

Composition

Parametrisation

Quaternions

Homogeneous Transformations

Matlab Examples for Homogeneous Transformations

Some questions for you

Fundamental Rotation Matrices

Composition of Rotations

Parametrisation of Rotations

Axis–Angle Representation

Quaternions

Homogeneous Transformations



Quaternions

- Consider a real number $x \in \mathbb{R}$. What is its inverse?

$$\frac{1}{x} \cdot x = 1 \implies x^{-1} = \frac{1}{x}.$$

Quaternions

- Consider a real number $x \in \mathbb{R}$. What is its inverse?

$$\frac{1}{x} \cdot x = 1 \implies x^{-1} = \frac{1}{x}.$$

- Consider an element of $\mathbb{R}^2$, $\mathbf{x} = \begin{bmatrix} a \\ b \end{bmatrix}$. Can we find its inverse? Yes, by using complex numbers. Let's define $z = a + ib$, where i is the imaginary unit. We can introduce the ****complex conjugate**** $z^* = a - ib$ and consider the number $z_1 = \frac{z^*}{|z|^2}$. We have:

$$z \cdot z_1 = \frac{z \cdot z^*}{|z|^2} = \frac{a^2 + b^2}{a^2 + b^2} = 1. \text{ Hence, } z^{-1} = z_1.$$

Quaternions

- Consider a real number $x \in \mathbb{R}$. What is its inverse?

$$\frac{1}{x} \cdot x = 1 \implies x^{-1} = \frac{1}{x}.$$

- Consider an element of $\mathbb{R}^2$, $\mathbf{x} = \begin{bmatrix} a \\ b \end{bmatrix}$. Can we find its inverse? Yes, by using complex numbers. Let's define $z = a + ib$, where i is the imaginary unit. We can introduce the ****complex conjugate**** $z^* = a - ib$ and consider the number $z_1 = \frac{z^*}{|z|^2}$. We have:

$$z \cdot z_1 = \frac{z \cdot z^*}{|z|^2} = \frac{a^2 + b^2}{a^2 + b^2} = 1. \text{ Hence, } z^{-1} = z_1.$$

- We cannot repeat the same trick with $\mathbb{R}^3$, but...

Quaternions

- Consider a real number $x \in \mathbb{R}$. What is its inverse?

$$\frac{1}{x} \cdot x = 1 \implies x^{-1} = \frac{1}{x}.$$

- Consider an element of $\mathbb{R}^2$, $\mathbf{x} = \begin{bmatrix} a \\ b \end{bmatrix}$. Can we find its inverse? Yes, by using complex numbers. Let's define $z = a + ib$, where i is the imaginary unit. We can introduce the ****complex conjugate**** $z^* = a - ib$ and consider the number $z_1 = \frac{z^*}{|z|^2}$. We have:

$$z \cdot z_1 = \frac{z \cdot z^*}{|z|^2} = \frac{a^2 + b^2}{a^2 + b^2} = 1. \text{ Hence, } z^{-1} = z_1.$$

- We cannot repeat the same trick with $\mathbb{R}^3$, but...
- We can with $\mathbb{R}^4$, by using *quaternions*.

Definition

Definition of a Complex Number

- ▶ Let 1 and i be the basis elements.
- ▶ The following axiom holds: $i^2 = -1$.
- ▶ A complex number is defined as an element of a vector space over the reals, with the form $z = x + iy$, where $x, y \in \mathbb{R}$.

Definition

Definition of a Complex Number

- ▶ Let 1 and i be the basis elements.
- ▶ The following axiom holds: $i^2 = -1$.
- ▶ A complex number is defined as an element of a vector space over the reals, with the form $z = x + iy$, where $x, y \in \mathbb{R}$.

Definition of a Quaternion

- ▶ Let 1 and i, j, k be the basis elements.
- ▶ The following axioms hold: $i^2 = j^2 = k^2 = ijk = -1$.
- ▶ A quaternion is defined as an element of a vector space over the reals, with the form $q = q_0 + q_1i + q_2j + q_3k$.
- ▶ A shorthand notation distinguishes the ****scalar part**** q_0 and the ****vector part**** $\mathbf{q} = q_1i + q_2j + q_3k$: $q = q_0 + \mathbf{q}$. An equivalent notation is $[q_0, \mathbf{q}]$.

Background on Vector Products

Scalar Product (Dot Product)

Given two vectors $\mathbf{u} = u_1\mathbf{i} + u_2\mathbf{j} + u_3\mathbf{k}$ and $\mathbf{v} = v_1\mathbf{i} + v_2\mathbf{j} + v_3\mathbf{k}$, the *scalar product* (or dot product) $\mathbf{u} \cdot \mathbf{v}$ is the real number

$\mathbf{u} \cdot \mathbf{v} = u_1v_1 + u_2v_2 + u_3v_3$. If θ is the angle between the vectors, we have: $\mathbf{u} \cdot \mathbf{v} = |\mathbf{u}| |\mathbf{v}| \cos \theta$. This operation represents the projection of one vector onto the other and is *commutative*.

Background on Vector Products

Scalar Product (Dot Product)

Given two vectors $\mathbf{u} = u_1\mathbf{i} + u_2\mathbf{j} + u_3\mathbf{k}$ and $\mathbf{v} = v_1\mathbf{i} + v_2\mathbf{j} + v_3\mathbf{k}$, the *scalar product* (or dot product) $\mathbf{u} \cdot \mathbf{v}$ is the real number

$\mathbf{u} \cdot \mathbf{v} = u_1v_1 + u_2v_2 + u_3v_3$. If θ is the angle between the vectors, we have: $\mathbf{u} \cdot \mathbf{v} = |\mathbf{u}| |\mathbf{v}| \cos \theta$. This operation represents the projection of one vector onto the other and is *commutative*.

Vector Product (Cross Product)

Given two vectors $\mathbf{u} = u_1\mathbf{i} + u_2\mathbf{j} + u_3\mathbf{k}$ and $\mathbf{v} = v_1\mathbf{i} + v_2\mathbf{j} + v_3\mathbf{k}$, the *vector product* (or cross product) $\mathbf{s} = \mathbf{u} \times \mathbf{v}$ is a vector orthogonal to both $\mathbf{u}$ and $\mathbf{v}$, whose magnitude is given by $|\mathbf{s}| = |\mathbf{u}| |\mathbf{v}| \sin \theta$. We can show that:

$$\mathbf{s} = \mathbf{u} \times \mathbf{v} = s_1\mathbf{i} + s_2\mathbf{j} + s_3\mathbf{k}$$

$$\implies \begin{cases} s_1 = u_2v_3 - u_3v_2 \\ s_2 = u_3v_1 - u_1v_3 \\ s_3 = u_1v_2 - u_2v_1 \end{cases}$$

The vector product is *anti-commutative*: $\mathbf{u} \times \mathbf{v} = -\mathbf{v} \times \mathbf{u}$.



Basic Facts on Multiplication

By applying the axiom $i^2 = j^2 = k^2 = ijk = -1$, we can derive the following multiplication rules for the basis elements:

$$ij = k, \quad ji = -k$$

$$jk = i, \quad kj = -i$$

$$ki = j, \quad ik = -j$$

These rules for multiplying the basis vectors are the same as those for the *vector cross product*.



Quaternion Product

We can use the shorthand vector notation to compute the product of two quaternions, $p = p_0 + \mathbf{p}$ and $q = q_0 + \mathbf{q}$:

$$\begin{aligned}pq &= (p_0 + \mathbf{p})(q_0 + \mathbf{q}) \\ &= p_0q_0 + p_0\mathbf{q} + q_0\mathbf{p} + \mathbf{p}\mathbf{q}\end{aligned}$$



Quaternion Product

We can use the shorthand vector notation to compute the product of two quaternions, $p = p_0 + \mathbf{p}$ and $q = q_0 + \mathbf{q}$:

$$\begin{aligned}pq &= (p_0 + \mathbf{p})(q_0 + \mathbf{q}) \\ &= p_0 q_0 + p_0 \mathbf{q} + q_0 \mathbf{p} + \mathbf{p} \mathbf{q}\end{aligned}$$

Using the rules of quaternion multiplication, the product of the two vector parts simplifies as $\mathbf{p} \mathbf{q} = -\mathbf{p} \cdot \mathbf{q} + \mathbf{p} \times \mathbf{q}$. Thus, the full product is:

$$pq = (p_0 q_0 - \mathbf{p} \cdot \mathbf{q}) + (p_0 \mathbf{q} + q_0 \mathbf{p} + \mathbf{p} \times \mathbf{q})$$



Quaternion Product

We can use the shorthand vector notation to compute the product of two quaternions, $p = p_0 + \mathbf{p}$ and $q = q_0 + \mathbf{q}$:

$$\begin{aligned} pq &= (p_0 + \mathbf{p})(q_0 + \mathbf{q}) \\ &= p_0 q_0 + p_0 \mathbf{q} + q_0 \mathbf{p} + \mathbf{p} \mathbf{q} \end{aligned}$$

Using the rules of quaternion multiplication, the product of the two vector parts simplifies as $\mathbf{p} \mathbf{q} = -\mathbf{p} \cdot \mathbf{q} + \mathbf{p} \times \mathbf{q}$. Thus, the full product is:

$$pq = (p_0 q_0 - \mathbf{p} \cdot \mathbf{q}) + (p_0 \mathbf{q} + q_0 \mathbf{p} + \mathbf{p} \times \mathbf{q})$$

The resulting quaternion has a scalar part of $p_0 q_0 - \mathbf{p} \cdot \mathbf{q}$ and a vector part of $p_0 \mathbf{q} + q_0 \mathbf{p} + \mathbf{p} \times \mathbf{q}$.

The quaternion product is *associative*, but it is *not commutative* ($pq \neq qp$).

Quaternion Conjugate

Definition of a Quaternion Conjugate

$$q = q_0 + q_1i + q_2j + q_3k \implies q^* = q_0 - q_1i - q_2j - q_3k$$

By applying this definition and carrying out the computations, we get:

$$\begin{aligned} qq^* &= (q_0 + \mathbf{q})(q_0 - \mathbf{q}) \\ &= q_0^2 - q_0\mathbf{q} + q_0\mathbf{q} - \mathbf{q}^2 \\ &= q_0^2 - (-\mathbf{q} \cdot \mathbf{q} + \mathbf{q} \times \mathbf{q}) \\ &= q_0^2 + |\mathbf{q}|^2 \\ &= q_0^2 + q_1^2 + q_2^2 + q_3^2 \end{aligned}$$

Definition of Quaternion Length

The quantity $|q| = \sqrt{qq^*} = \sqrt{q_0^2 + q_1^2 + q_2^2 + q_3^2}$ is defined as the *quaternion length* or *norm*.

Quaternion Inverse

Based on the definition above, a quaternion (except 0) has a *unique* inverse $q^{-1} = \frac{q^*}{|q|^2}$:

$$q \frac{q^*}{|q|^2} = \frac{qq^*}{|q|^2} = 1.$$

The relation above simplifies even further when the quaternion has unit-length.



To Complex Numbers and Back

We know that a complex number z can be expressed in its Euler form as:

$$z = \rho (\cos (\theta) + j \sin (\theta))$$

To Complex Numbers and Back

We know that a complex number z can be expressed in its Euler form as:

$$z = \rho (\cos (\theta) + j \sin (\theta))$$

When we multiply two complex numbers in Euler notation, we get:

$$z_1 z_2 = \rho_1 (\cos (\theta_1) + j \sin (\theta_1)) \rho_2 (\cos (\theta_2) + j \sin (\theta_2)) = \rho_1 \rho_2 (\cos (\theta_1 + \theta_2) + j \sin (\theta_1 + \theta_2))$$

To Complex Numbers and Back

We know that a complex number z can be expressed in its Euler form as:

$$z = \rho (\cos (\theta) + j \sin (\theta))$$

When we multiply two complex numbers in Euler notation, we get:

$$z_1 z_2 = \rho_1 (\cos (\theta_1) + j \sin (\theta_1)) \rho_2 (\cos (\theta_2) + j \sin (\theta_2)) = \rho_1 \rho_2 (\cos (\theta_1 + \theta_2) + j \sin (\theta_1 + \theta_2))$$

Multiplication by Unit-Norm Complex Numbers

When we multiply a complex number by a unit-norm (or unit-modulus) complex number, the effect is a rotation.

Multiplying a complex number by its complex conjugate "undoes" the rotation, aligning the result with the real axis.



Unit-Length Quaternions

Consider a *unit-length quaternion*: $q = [\eta, \epsilon]$, where η is the scalar part

and $\epsilon = \begin{bmatrix} \epsilon_x \\ \epsilon_y \\ \epsilon_z \end{bmatrix}$ is the vector part.

We can express a unit-length quaternion as:

$$q = \left[\cos\left(\frac{\theta}{2}\right), \sin\left(\frac{\theta}{2}\right) \mathbf{k} \right]$$

where $\mathbf{k}$ is a unit vector defined as:

$$\mathbf{k} = \frac{\epsilon}{|\epsilon|}$$

$$\theta = 2\text{atan2}(|\epsilon|, \eta)$$

Key Fact

The unit-length quaternion $q = \left[\cos\left(\frac{\theta}{2}\right), \sin\left(\frac{\theta}{2}\right) \mathbf{k} \right]$ represents a rotation of angle θ about the axis defined by the unit vector $\mathbf{k}$.

Why is this the case?

Quaternions and Rotations (1)

Let $q = [\eta, \epsilon]$ be a unit quaternion and $p = [0, \mathbf{p}]$ be a pure quaternion representing a vector $\mathbf{p}$.

The rotated vector $\mathbf{p}'$ can be computed using the conjugation formula: $p' = qpq^*$. This operation yields a pure quaternion of the form $[0, \mathbf{p}']$.



Quaternions and Rotations (1)

Let $q = [\eta, \epsilon]$ be a unit quaternion and $p = [0, \mathbf{p}]$ be a pure quaternion representing a vector $\mathbf{p}$.

The rotated vector $\mathbf{p}'$ can be computed using the conjugation formula:

$p' = qpq^*$. This operation yields a pure quaternion of the form $[0, \mathbf{p}']$.

After a considerable amount of computation, we find that the vector part $\mathbf{p}'$ is given by:

$$\mathbf{p}' = (\eta^2 - |\epsilon|^2)\mathbf{p} + 2\eta(\epsilon \times \mathbf{p}) + 2(\epsilon \cdot \mathbf{p})\epsilon$$

Quaternions and Rotations (2)

By substituting $\eta = \cos\left(\frac{\theta}{2}\right)$ and $\epsilon = \sin\left(\frac{\theta}{2}\right) \mathbf{k}$, and using the identity $|\epsilon|^2 = \sin^2\left(\frac{\theta}{2}\right)$, the expression simplifies to:

$$\begin{aligned}\mathbf{p}' &= \left(\cos^2\left(\frac{\theta}{2}\right) - \sin^2\left(\frac{\theta}{2}\right) \right) \mathbf{p} + 2 \cos\left(\frac{\theta}{2}\right) \sin\left(\frac{\theta}{2}\right) (\mathbf{k} \times \mathbf{p}) + \\ &\quad + 2 \sin^2\left(\frac{\theta}{2}\right) (\mathbf{k} \cdot \mathbf{p}) \mathbf{k} \\ &= \cos(\theta) \mathbf{p} + \sin(\theta) (\mathbf{k} \times \mathbf{p}) + (1 - \cos(\theta)) (\mathbf{k} \cdot \mathbf{p}) \mathbf{k}\end{aligned}$$

The formula:

$$\mathbf{p}' = \cos(\theta) \mathbf{p} + \sin(\theta) (\mathbf{k} \times \mathbf{p}) + (1 - \cos(\theta)) (\mathbf{k} \cdot \mathbf{p}) \mathbf{k}$$

is the **Rodrigues' rotation formula**. It describes the rotation of a vector $\mathbf{p}$ by an angle θ about the axis $\mathbf{k}$.

Quaternions and Rotations (3)

The Rodrigues rotation formula is equivalent to applying a rotation matrix $R_{\mathbf{k}}(\theta)$:

$$\mathbf{p}' = R_{\mathbf{k}}(\theta)\mathbf{p}$$

where $R_{\mathbf{k}}(\theta)$ is the matrix corresponding to a rotation of angle θ about the axis $\mathbf{k}$:

$$R_{\mathbf{k}}(\theta) = \begin{bmatrix} k_x^2(1 - c_\theta) + c_\theta & k_x k_y(1 - c_\theta) - k_z s_\theta & k_x k_z(1 - c_\theta) + k_y s_\theta \\ k_x k_y(1 - c_\theta) + k_z s_\theta & k_y^2(1 - c_\theta) + c_\theta & k_y k_z(1 - c_\theta) - k_x s_\theta \\ k_x k_z(1 - c_\theta) - k_y s_\theta & k_y k_z(1 - c_\theta) + k_x s_\theta & k_z^2(1 - c_\theta) + c_\theta \end{bmatrix}$$

where $c_\theta = \cos(\theta)$ and $s_\theta = \sin(\theta)$.

Summary

Given a unit-length quaternion $q = [\eta, \epsilon] = [\cos(\frac{\theta}{2}), \sin(\frac{\theta}{2}) \mathbf{k}]$, we can associate it with a rotation of angle θ about the unit vector $\mathbf{k}$.

The parameters are related by:

$$\mathbf{k} = \frac{\epsilon}{|\epsilon|}$$

$$\theta = 2 \operatorname{atan2}(|\epsilon|, \eta)$$

The corresponding rotation matrix is given by:

$$R_{\mathbf{k}}(\theta) = \begin{bmatrix} k_x^2(1 - c_\theta) + c_\theta & k_x k_y(1 - c_\theta) - k_z s_\theta & k_x k_z(1 - c_\theta) + k_y s_\theta \\ k_x k_y(1 - c_\theta) + k_z s_\theta & k_y^2(1 - c_\theta) + c_\theta & k_y k_z(1 - c_\theta) - k_x s_\theta \\ k_x k_z(1 - c_\theta) - k_y s_\theta & k_y k_z(1 - c_\theta) + k_x s_\theta & k_z^2(1 - c_\theta) + c_\theta \end{bmatrix}$$

where $c_\theta = \cos(\theta)$ and $s_\theta = \sin(\theta)$.

The parameters of a unit-length quaternion correspond to three degrees of freedom, which can be expressed as:

- ▶ four numbers: $\eta, \epsilon_x, \epsilon_y, \epsilon_z$, with the constraint $\eta^2 + \epsilon_x^2 + \epsilon_y^2 + \epsilon_z^2 = 1$.
- ▶ four numbers: θ, k_x, k_y, k_z , with the constraint $k_x^2 + k_y^2 + k_z^2 = 1$.

From Rotation Matrix to Quaternions

- Given a rotation matrix $R = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix}$, we can solve the inverse problem:

$$\eta = \frac{1}{2} \sqrt{r_{11} + r_{22} + r_{33} + 1}$$

$$\epsilon = \frac{1}{2} \begin{bmatrix} \text{sgn}(r_{32} - r_{23}) \sqrt{r_{11} - r_{22} - r_{33} + 1} \\ \text{sgn}(r_{13} - r_{31}) \sqrt{r_{22} - r_{33} - r_{11} + 1} \\ \text{sgn}(r_{21} - r_{12}) \sqrt{r_{33} - r_{11} - r_{22} + 1} \end{bmatrix} \quad \text{where}$$

$$\text{sgn}(x) = \begin{cases} 1 & x \geq 0 \\ -1 & x < 0 \end{cases}$$



Composition of Rotations

Consider two rotations associated with the unit-length quaternions q_0 and q_1 .

The first rotation transforms a vector $\mathbf{p}$ into $\mathbf{p}'$, and the second transforms $\mathbf{p}'$ into $\mathbf{p}''$.

We can represent these rotations using pure quaternions $p = [0, \mathbf{p}]$, $p' = [0, \mathbf{p}']$, and $p'' = [0, \mathbf{p}'']$.

We have:

$$\begin{aligned} p' &= q_0 p q_0^* \\ p'' &= q_1 p' q_1^* \\ &= q_1 (q_0 p q_0^*) q_1^* \\ &= (q_1 q_0) p (q_0^* q_1^*) \\ &= (q_1 q_0) p (q_1 q_0)^* \end{aligned}$$

The last step holds because the conjugate of a product is the product of the conjugates in reverse order, i.e., $(q_1 q_0)^* = q_0^* q_1^*$.

Therefore, *the composition of two rotations is equivalent to the product of their corresponding quaternions*. This demonstrates the power and elegance of quaternions for representing rotations.

Quaternions: Operations and Advantages

Quaternions provide efficient and elegant operations that correspond directly to those used with rotation matrices.

Operation	Rotation Matrices	Unit-Quaternions
Inversion	$R^{-1} = R^T$	$q^{-1} = (\eta, -\epsilon)$
Composition	$R = R_1 R_2$	$q = q_1 q_2$
Composition Formula		$q_1 q_2 = (\eta_1 \eta_2 - \epsilon_1 \cdot \epsilon_2, \eta_1 \epsilon_2 + \eta_2 \epsilon_1 + \epsilon_1 \times \epsilon_2)$

- **Inversion:** The inverse of a unit-length quaternion is its conjugate, which is found by simply negating the vector part. This is computationally much faster than transposing and inverting a matrix.
- **Composition:** The composition of two rotations is a simple quaternion product. The resulting quaternion represents the combined rotation. This is more efficient and numerically stable than composing rotation matrices.

In summary

- ▶ **Compact Representation:** Quaternions use four numbers to represent a rotation, which is more compact than the nine numbers of a rotation matrix.

In summary

- ▶ **Compact Representation:** Quaternions use four numbers to represent a rotation, which is more compact than the nine numbers of a rotation matrix.
- ▶ **Absence of Gimbal Lock:** Unlike Euler angles, quaternions provide a continuous and non-singular representation of rotations, completely avoiding the Gimbal Lock problem. This is a significant advantage in computer graphics and robotics.

In summary

- ▶ **Compact Representation:** Quaternions use four numbers to represent a rotation, which is more compact than the nine numbers of a rotation matrix.
- ▶ **Absence of Gimbal Lock:** Unlike Euler angles, quaternions provide a continuous and non-singular representation of rotations, completely avoiding the Gimbal Lock problem. This is a significant advantage in computer graphics and robotics.
- ▶ **Efficient Composition:** The composition of two rotations is achieved by a simple quaternion multiplication, which is computationally more efficient than matrix multiplication.

In summary

- ▶ **Compact Representation:** Quaternions use four numbers to represent a rotation, which is more compact than the nine numbers of a rotation matrix.
- ▶ **Absence of Gimbal Lock:** Unlike Euler angles, quaternions provide a continuous and non-singular representation of rotations, completely avoiding the Gimbal Lock problem. This is a significant advantage in computer graphics and robotics.
- ▶ **Efficient Composition:** The composition of two rotations is achieved by a simple quaternion multiplication, which is computationally more efficient than matrix multiplication.
- ▶ **Direct Interpolation:** Quaternions allow for smooth, constant-velocity interpolation between two rotations (spherical linear interpolation or slerp), which is crucial for animations.

In summary

- ▶ **Compact Representation:** Quaternions use four numbers to represent a rotation, which is more compact than the nine numbers of a rotation matrix.
- ▶ **Absence of Gimbal Lock:** Unlike Euler angles, quaternions provide a continuous and non-singular representation of rotations, completely avoiding the Gimbal Lock problem. This is a significant advantage in computer graphics and robotics.
- ▶ **Efficient Composition:** The composition of two rotations is achieved by a simple quaternion multiplication, which is computationally more efficient than matrix multiplication.
- ▶ **Direct Interpolation:** Quaternions allow for smooth, constant-velocity interpolation between two rotations (spherical linear interpolation or *slerp*), which is crucial for animations.
- ▶ **Non-Uniqueness:** A single rotation can be represented by two quaternions, q and $-q$. While this is a form of redundancy, it is a minor issue that does not demean the practical benefits of quaternions.

Matlab Implementation of rotations

Still on Rotation Matrices

Composition

Parametrisation

Quaternions

Homogeneous Transformations

Matlab Examples for Homogeneous Transformations

Some questions for you

Fundamental Rotation Matrices

Composition of Rotations

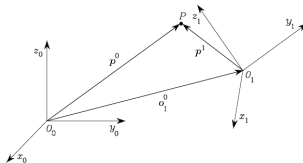
Parametrisation of Rotations

Axis–Angle Representation

Quaternions

Homogeneous Transformations

Position of a Frame in 3D

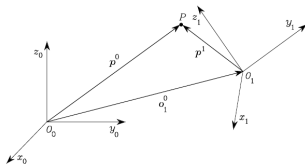


- ▶ We established that a rigid body's pose is defined by the position and orientation of a frame attached to it.
- ▶ We then considered only rotation, ignoring translation (with the origins of the frames being coincident).
- ▶ We now return to the general case, where both rotation and translation are present.
- ▶ A point $\mathbf{p}^1$ in frame $\{1\}$ can be expressed in frame $\{0\}$ as:

$$\mathbf{p}^0 = \mathbf{o}_1^0 + R_1^0 \mathbf{p}^1.$$



Direct and Inverted Coordinate Transformation



- The inverse relation of

$$\mathbf{p}^0 = \mathbf{o}_1^0 + R_1^0 \mathbf{p}^1$$

is found by isolating $\mathbf{p}^1$:

$$\begin{aligned} R_1^0 \mathbf{p}^1 &= \mathbf{p}^0 - \mathbf{o}_1^0 \\ \mathbf{p}^1 &= (R_1^0)^{-1} (\mathbf{p}^0 - \mathbf{o}_1^0) \\ &= R_1^{0T} (\mathbf{p}^0 - \mathbf{o}_1^0) \\ &= R_0^1 (\mathbf{p}^0 - \mathbf{o}_1^0) \\ &= R_0^1 \mathbf{p}^0 - R_0^1 \mathbf{o}_1^0. \end{aligned}$$

- This representation is rather cumbersome, as the inverse involves



Homogeneous Coordinates (1)

- ▶ We need a compact way to represent both rotation and translation.
- ▶ The technique is simple: we create the *homogeneous representation* $\tilde{\mathbf{p}}$ of $\mathbf{p}$ by appending a coordinate with a value of 1.

$$\tilde{\mathbf{p}} = \begin{bmatrix} \mathbf{p} \\ 1 \end{bmatrix}$$

- ▶ We can then construct a 4×4 matrix, called the *homogeneous transformation matrix*:

$$A_1^0 = \begin{bmatrix} R_1^0 & \mathbf{o}_1^0 \\ \mathbf{0}^T & 1 \end{bmatrix}$$

- ▶ The transformation of a point from frame $\{1\}$ to frame $\{0\}$ can now be written as a single matrix-vector multiplication:

$$\tilde{\mathbf{p}}^0 = A_1^0 \tilde{\mathbf{p}}^1.$$

Homogeneous Coordinates (2)

- ▶ A key advantage is that the coordinate transformation is a simple matrix multiplication:

$$\tilde{\mathbf{p}}^0 = A_1^0 \tilde{\mathbf{p}}^1.$$

- ▶ The inverse transformation is found by simply inverting the matrix:

$$\begin{aligned}\tilde{\mathbf{p}}^1 &= A_0^1 \tilde{\mathbf{p}}^0 \\ &= (A_1^0)^{-1} \tilde{\mathbf{p}}^0.\end{aligned}$$

- ▶ The inverse homogeneous transformation matrix can be derived as:

$$A_0^1 = (A_1^0)^{-1} = \begin{bmatrix} R_1^{0T} & -R_1^{0T} \mathbf{o}_1^0 \\ \mathbf{0}^T & 1 \end{bmatrix}.$$

Homogeneous Coordinates (3)

- ▶ Unlike rotation matrices, homogeneous transformations are not orthogonal. Therefore, for a general homogeneous matrix A , we have $A^{-1} \neq A^T$.
- ▶ The homogeneous formalism allows us to generalize the properties of the rotation group to include translations.
- ▶ For example, a sequence of transformations in the current axes can be written as a simple matrix product:

$$\tilde{\mathbf{p}}^0 = A_1^0 A_2^1 \dots A_n^{n-1} \tilde{\mathbf{p}}^n.$$

- ▶ The set of all homogeneous transformations forms a group called the **special Euclidean group**, denoted as $SE(3)$. This group is the semi-direct product $SE(3) = \mathbb{R}^3 \rtimes SO(3)$.

Matlab Implementation of rotations

Still on Rotation Matrices

Composition

Parametrisation

Quaternions

Homogeneous Transformations

Matlab Examples for Homogeneous Transformations

Some questions for you

Fundamental Rotation Matrices

Composition of Rotations

Parametrisation of Rotations

Axis–Angle Representation

Quaternions

Homogeneous Transformations

Homogeneous Transformation

Similarly, we can create homogeneous transformation matrices:

$$A_1^0 = \begin{bmatrix} R(\phi, \theta, \psi) & O_1^0 \\ O & 1 \end{bmatrix}$$

The corresponding Matlab listing is:

homogeneousTrans

```
1 %homogeneous transformation
2 function [T] = homogeneousTrans(phi , theta , psi ,Ov)
3 R = eulerRot(phi , theta , psi );
4
5 T = [R, Ov;
6      zeros(1,3) , 1]
7 end
```

Homogeneous Transformation – Test

The Matlab script to test the function is the following:

scriptDrawHom

```

1  close all;
2
3  view(3);
4  daspect([1 1 1]);
5  plot3([0,1],[0,0],[0,0], 'Color',[1,0,0]);
6  hold on
7  plot3([0,0],[0,1],[0,0], 'Color',[0,1,0]);
8  plot3([0,0],[0,0],[0,1], 'Color',[0,0,1]);
9
10 T = homogeneousTrans(0, 0, pi/4,[1;1;1]);
11 O1 = T*[0;0;0;1];
12
13 v1 = T*[1;0;0;1]
14 v2 = T*[0;1;0;1]
15 v3 = T*[0;0;1;1]
16
17 plot3([O1(1),v1(1)],[O1(2),v1(2)],[O1(3),v1(3)], 'Color',[1,0,0], 'LineStyle','—');
18 plot3([O1(1),v2(1)],[O1(2),v2(2)],[O1(3),v2(3)], 'Color',[0,1,0], 'LineStyle','—');
19 plot3([O1(1),v3(1)],[O1(2),v3(2)],[O1(3),v3(3)], 'Color',[0,0,1], 'LineStyle','—');

```

Homogeneous Transformation – Test

We can equivalently use built-in Matlab functions:

scriptDrawHom1

```
1  close all;
2
3  view(3);
4  daspect([1 1 1]);
5  plot3([0,1],[0,0],[0,0], 'Color',[1,0,0]);
6  hold on
7  plot3([0,0],[0,1],[0,0], 'Color',[0,1,0]);
8  plot3([0,0],[0,0],[0,1], 'Color',[0,0,1]);
9
10 T = makehgtform('translate',[1,1,1], 'zrotate',pi/4)
11 O1 = T*[0;0;0;1];
12
13 v1 = T*[1;0;0;1]
14 v2 = T*[0;1;0;1]
15 v3 = T*[0;0;1;1]
16
17 plot3([O1(1),v1(1)],[O1(2),v1(2)],[O1(3),v1(3)], 'Color',[1,0,0], 'LineStyle','—');
18 plot3([O1(1),v2(1)],[O1(2),v2(2)],[O1(3),v2(3)], 'Color',[0,1,0], 'LineStyle','—');
19 plot3([O1(1),v3(1)],[O1(2),v3(2)],[O1(3),v3(3)], 'Color',[0,0,1], 'LineStyle','—');
```

A Function to Draw Reference Frames

I have downloaded a community file to draw axis frames.

scriptTestTriad

```
1  close all
2  %Create some axis
3  axs = axes;
4
5  %Set 3D view
6  view(3);
7
8  %Set aspect ratio
9  daspect([1 1 1]);
10
11 %Draw a first reference frame that coincides with the current axis
12 h = triad('Parent', axs);
13
14 %Now draw a second reference frame obtained with a translation and a
15 %rotation about x
16 h1=triad('Parent',h,'Matrix',makehgtform('translate',[1,1,1],'xrotate',pi/4),'
           linewidth',3,'linestyle','—')
```



Matlab Transformation Objects

Matlab provides an object to create a transformation, link it to the current axis, and then define child objects associated with it that are transformed accordingly.

script TransformTest

```
1 clear all;
2 close all;
3 %Define the axis
4 axs=axes('XLim',[-1.5 1.5], 'YLim',[-1.5 1.5], 'ZLim',[-1.5 1.5]);
5 view(3)
6 grid on;
7 % Create an empty transformation referred
8 % to the axis just created
9 t = hgtransform('Parent',axs);
10 %Create a cylinder
11 [x,y,z] = cylinder([1,1],100);
12
13 h = surface(x,y,z, 'FaceColor', 'red');
14 %Associate the cylinder with the
15 %transformation object
16 set(h, 'Parent',t);
17 %Wait for a key to be pressed
18 pause
19 %create a rotation matrix
20 Rz = makehgtform('xrotate',2*pi/3);
21 %apply to the transformation object and to its children
22 set(t, 'Matrix',Rz);
23 %Update the children objects
24 drawnow;
```

Matlab Transformation: Example with Frames

Let us consider a transformation with frames.

script TransformTriads

```

1  clear all;
2  close all;
3  %Define the axis
4  axs=axes('XLim',[-2.5 1.5], 'YLim',[-2.5 2.5], 'ZLim',[-2.5 2.5]);
5  view(3)
6  grid on;
7  % Create an empty transformation referred
8  % to the axis just created
9  t = hgtransform('Parent',axs);
10 h = triad('Parent', t);
11 %Now draw a second reference frame obtained with a translation and a
12 %rotation about x
13 h1=triad('Parent',h,'Matrix',makehgtform('translate',[1,1,1], 'xrotate',pi/4))
14 plot3([0,1],[0,1],[0,1], 'Color',[1,0,0], 'Parent',h, 'linewidth',3, 'linestyle',
        '—');
15 %Wait for a key to be pressed
16 pause
17 for angle = 0:0.5:360,
18 %create a rotation matrix
19 Rz = makehgtform('zrotate',deg2rad(angle));
20 %apply to the transformation object and to its children
21 set(t,'Matrix',Rz);
22 %Update the children objects
23 drawnow;
24 end

```

Outline

Matlab Implementation of rotations

Still on Rotation Matrices

- Composition

- Parametrisation

- Quaternions

Homogeneous Transformations

- Matlab Examples for Homogeneous Transformations

Some questions for you

- Fundamental Rotation Matrices

- Composition of Rotations

- Parametrisation of Rotations

- Axis–Angle Representation

- Quaternions

- Homogeneous Transformations

Outline

Matlab Implementation of rotations

Still on Rotation Matrices

Composition

Parametrisation

Quaternions

Homogeneous Transformations

Matlab Examples for Homogeneous Transformations

Some questions for you

Fundamental Rotation Matrices

Composition of Rotations

Parametrisation of Rotations

Axis–Angle Representation

Quaternions

Homogeneous Transformations

Question

Write the rotation matrix about the x-axis by an angle γ .

Answer

$$R_x(\gamma) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \gamma & -\sin \gamma \\ 0 & \sin \gamma & \cos \gamma \end{bmatrix}$$

Question

What is the difference between a rotation matrix defined with respect to the current axes versus the fixed axes?

Answer

- ▶ **Current axes:** each successive rotation is applied relative to the moving frame (post-multiplication).
- ▶ **Fixed axes:** rotations are applied relative to the original fixed frame (pre-multiplication).

Question

In Matlab, how would you implement the rotation about the y-axis?

Outline

Matlab Implementation of rotations

Still on Rotation Matrices

Composition

Parametrisation

Quaternions

Homogeneous Transformations

Matlab Examples for Homogeneous Transformations

Some questions for you

Fundamental Rotation Matrices

Composition of Rotations

Parametrisation of Rotations

Axis–Angle Representation

Quaternions

Homogeneous Transformations

Question

If a vector is rotated first by $R_y(\varphi)$ and then by $R_z(\theta)$, what is the resulting rotation matrix?

Answer

$$R = R_y(\varphi)R_z(\theta)$$

Question

Why does the order of multiplication matter in rotation compositions?

Answer

Rotation matrices do not generally commute:

$$R_a R_b \neq R_b R_a.$$

Thus, changing the order changes the final orientation.



Question

For rotations about fixed axes, why do we pre-multiply matrices instead of post-multiplying?

Answer

Because each new rotation is applied relative to the original fixed frame, the correct composition is obtained by pre-multiplication.

Outline

Matlab Implementation of rotations

Still on Rotation Matrices

Composition

Parametrisation

Quaternions

Homogeneous Transformations

Matlab Examples for Homogeneous Transformations

Some questions for you

Fundamental Rotation Matrices

Composition of Rotations

Parametrisation of Rotations

Axis–Angle Representation

Quaternions

Homogeneous Transformations

Question

How many independent parameters fully define a 3D rotation?
Why?

Answer

3 independent parameters. A rotation matrix is 3×3 (9 values) but orthogonality and unit determinant reduce it to 3 degrees of freedom.

Question

What are Euler angles? Give an example commonly used in aeronautics.

Answer

Euler angles are a sequence of 3 rotations about axes. Example: roll (ψ), pitch (θ), yaw (φ) with fixed axes:

$$R = R_z(\varphi)R_y(\theta)R_x(\psi)$$

Question

What is gimbal lock in the context of RPY angles?

Answer

When pitch $\theta = \pm\pi/2$, roll and yaw axes align, causing a loss of one degree of freedom. Different angle sets map to the same rotation.

Outline

Matlab Implementation of rotations

Still on Rotation Matrices

Composition

Parametrisation

Quaternions

Homogeneous Transformations

Matlab Examples for Homogeneous Transformations

Some questions for you

Fundamental Rotation Matrices

Composition of Rotations

Parametrisation of Rotations

Axis–Angle Representation

Quaternions

Homogeneous Transformations

Question

How is a 3D rotation represented in axis–angle form?

Answer

By a unit vector $k = (k_x, k_y, k_z)$ and an angle θ . The rotation is expressed as $R_k(\theta)$.

Question

State Rodrigues' rotation formula.

Answer

$$p' = \cos(\theta)p + \sin(\theta)(k \times p) + (1 - \cos(\theta))(k \cdot p)k$$

Question

Why does axis–angle parametrisation avoid gimbal lock?

Answer

Because it locally represents rotations without singularities. Unlike Euler angles, no two axes can align to cause a degree-of-freedom loss.

Outline

Matlab Implementation of rotations

Still on Rotation Matrices

Composition

Parametrisation

Quaternions

Homogeneous Transformations

Matlab Examples for Homogeneous Transformations

Some questions for you

Fundamental Rotation Matrices

Composition of Rotations

Parametrisation of Rotations

Axis–Angle Representation

Quaternions

Homogeneous Transformations

Question

Define a quaternion and identify its scalar and vector parts.

Answer

$$q = q_0 + q_1i + q_2j + q_3k = [q_0, \mathbf{q}]$$

- ▶ q_0 : scalar part.
- ▶ $\mathbf{q} = (q_1, q_2, q_3)$: vector part.

Question

What is the quaternion conjugate and how is it used to compute the inverse?

Answer

Conjugate: $q^* = q_0 - q_1i - q_2j - q_3k$.

Inverse: $q^{-1} = q^*/|q|^2$. For unit quaternions, $q^{-1} = q^*$.

Question

Show how the quaternion product of two unit quaternions corresponds to the composition of two rotations.

Answer

If q_1, q_2 represent rotations, then their composition is $q = q_1 q_2$.
Applying it to vector p :

$$p' = q_1(q_2 p q_2^*) q_1^* = (q_1 q_2) p (q_1 q_2)^*.$$

Question

Give two advantages of using quaternions over Euler angles in robotics.

Answer

- ▶ Avoid gimbal lock (non-singular representation).
- ▶ Efficient composition and interpolation (slerp).

Outline

Matlab Implementation of rotations

Still on Rotation Matrices

- Composition

- Parametrisation

- Quaternions

Homogeneous Transformations

- Matlab Examples for Homogeneous Transformations

Some questions for you

- Fundamental Rotation Matrices

- Composition of Rotations

- Parametrisation of Rotations

- Axis–Angle Representation

- Quaternions

- Homogeneous Transformations**



Question

What is the purpose of homogeneous coordinates in robotics?

Answer

They allow representation of both rotation and translation in a single 4×4 matrix.

Question

Write the general form of a homogeneous transformation matrix.

Answer

$$A = \begin{bmatrix} R & o \\ 0^T & 1 \end{bmatrix}$$

where R is a rotation matrix and o is a translation vector.

Question

How do you derive the inverse of a homogeneous transformation matrix?

Answer

$$A^{-1} = \begin{bmatrix} R^T & -R^T o \\ 0^T & 1 \end{bmatrix}$$

Question

What group do homogeneous transformations belong to?



Answer

They form the Special Euclidean group $SE(3) = \mathbb{R}^3 \rtimes SO(3)$.